Lab Notes

Advanced Measurement and Data Driven Modeling

Freja Vandeputte
John Lataire

Academic year
2025-2026

Bruface
Master in Electronics and Information Technology

Contents

Version: Monday 20th October, 2025 at 16:37.

Lab 1

Recursive least squares estimation with forgetting factor

1.1 Outline of the assignment and deliverable

In this assignment, a short recap of the recursive least-squares estimator for time-varying systems is given, followed by an exercise on a "time-varying" setup. The exercise requires to perform some measurements and some coding in Matlab. Furthermore, you are requested to complete a **report** on the exercise. In the report, you should summarize the main steps of the exercise, including figures and explanation of the relevant results, and provide the answers to the questions explicitly asked in the text of the exercise. The report will influence your grade for the Lab and it will guide the oral examination.

1.2 Goals

You will:

- Obtain time-varying measurements from an experimental setup.
- Familiarize yourself with the RLS estimation by implementing it in Matlab.
- Test the effects of the forgetting factor on the parameter estimation.
- Train scientific writing by delivering a report on the exercise.

1.3 Background

The recursive least-squares (RLS) estimation with forgetting factor can be used as a method to estimate the parameters of a time-varying system. At a given time N the time-varying system can be represented by:

$$y(N) = \sum_{i=0}^{n_b} b_i(N)u(N-i) - \sum_{i=1}^{n_a} a_i(N)y(N-i). \tag{1.1}$$

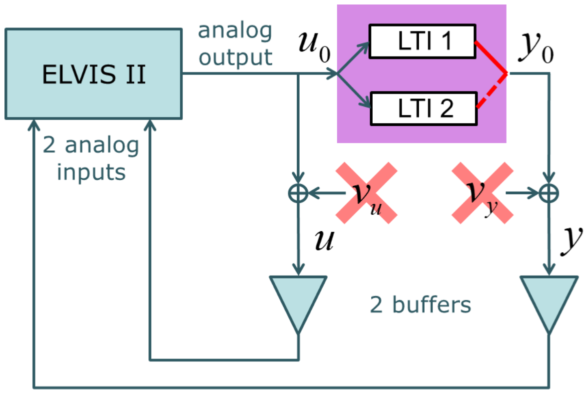

Figure 1.1: Schematics of the measurement setup, composed of an "LTV" system (in purple) connected to the ELVIS II.

or, in vector form, by:

$$y(N) = K(N)\theta(N). \tag{1.2}$$

Note that the parameter vector θ depends on N since we are dealing with a time-varying system and the parameter vector is different at all times. We are going to estimate $\theta(N)$ recursively. At every sample a new estimate of the parameter vector $\hat{\theta}(N)$ is computed, using the parameter vector computed at the previous sample $\hat{\theta}(N-1)$. From your theory notes, the RLS estimator with forgetting factor is calculated as follows:

$$\hat{\theta}(N) = \hat{\theta}(N-1) - \frac{P_{N-1}K^{\mathsf{T}}(N)(K(N)\hat{\theta}(N-1) - y(N))}{g + K(N)P_{N-1}K^{\mathsf{T}}(N)} \qquad N = 2, 3, \ldots \tag{1.3}$$

$$P_N = \frac{1}{g}(P_{N-1} - \frac{P_{N-1}K^{\mathsf{T}}(N)K(N)P_{N-1}}{g + K(N)P_{N-1}K^{\mathsf{T}}(N)}) \qquad N = 2, 3, \ldots \tag{1.4}$$

P_N represents the estimate of the covariance matrix at the time sample N. It is a $n_\theta \times n_\theta$ matrix where the diagonal elements are estimates of the variances of the parameters and the off-diagonal elements are the covariances. Notice that, to start the recursive procedure, we need to provide an initial value for the parameter vector $\hat{\theta}(1)$ and its covariance matrix $\hat{P}_1$.

1.4 Time-varying setup

In this exercise, we will estimate and track the parameters of a "time-varying" system by implementing an RLS algorithm with a forgetting-factor. The system considered, depicted in purple in Figure 1.1, is a SISO system with input $u_0(t)$ and output $y_0(t)$. The system is composed of two separate LTI sub-systems, and only one sub-system at a time can receive the input u_0 and generate the output

y_0. A manual switch on the surface of the device allows determining which of the sub-systems is being excited by the signal. If during the measurements the switch is used, then the system can be considered as a "time-varying" system, as the properties of the system will have changed during the measurements.

The system is physically connected to an ELVIS II platform, such that we can measure both the input and the output. We consider that the measurement noise on the input and the output are sufficiently small such that the bias due to the noise can be neglected. Furthermore, we consider that the system can be modeled by Equation (1.1), with $n_a = 2$ and $n_b = 1$.

1.5 Measurements

Generate an input signal $u_0(t)$ on Matlab as a white Gaussian noise signal with standard deviation 1, a sampling rate of 8 kHz and with number of samples $N = 64000$. Now, with the supervision of a teaching assistant, you can perform multiple measurements.

Task 1.1 *In LabView, apply the input signal and measure the output. While the data is measured, flip the switch a few times. Make sure that you perform several measurements, in which you change the number of times you flip the switch. For example, you could have a measurement in which you change the switch only once or twice and a measurement in which you change the switch several times.*

1.6 Time-varying RLS estimation

We will apply the time-varying RLS algorithm on the measurements and explore the effects of the forgetting factor.

Task 1.2 *Rewrite Equation (1.2), starting from (1.1). Define $n_a = 2$, $n_b = 1$ and $\theta(N)$ as $[b_0(N), b_1(N), a_1(N), a_2(N)]^\intercal$. Show the components of $K(N)$ explicitly.*

Task 1.3 *Implement the Matlab code to apply equations (1.3) and (1.4) on the measured data to estimate recursively the parameter vector $\hat{\theta}(N)$ and its covariance matrix $\hat{P}_N$. This can be done in a* `for` *loop ranging over the measurement points. Make sure to store the estimated parameters and the diagonal elements of the $\hat{P}_N$ matrix at every iteration. Comment your code sufficiently for later reference and do not hard code the variables, such that the algorithm can be tested for different parameters. Show the code on the report.*

Task 1.4 *To determine the initial value $\hat{\theta}(1)$, perform an LTI linear least squares estimation with only the first few measurements. Similarly, estimate $\hat{P}_1$ by applying the definition given in the lectures notes, using only the first few measurements. Explain the steps performed.*

Task 1.5 *Try different values for g and investigate if and how the parameters converge to a steady value or if they diverge. Which value did you pick for g? Motivate your choice.*

Once you find a satisfactory value for g that gives a smooth tracking of the parameters, we can look at the evolution of the parameters in time, and determine when the switch has been activated.

Task 1.6 *Plot the estimated parameters against time and visualize how they evolve. Compare their behavior to the diagonal components of the P matrix. In your plot add an indication of the time instants at which the switch has been activated, and of the time instants at which the parameters have converged. Did the estimate of the parameters converged to the true value? How long would this take?*

1.7 Transfer function

Task 1.7 *Pick two time instants in which the parameters have converged (from your best measurement), one for each LTI sub-systems. Extract the parameter vector at these two time instants and write them down in the report.*

Task 1.8 *Using the parameters just extracted, obtain and plot the frequency response of the two LTI sub-systems. (Hint: You can use the* `freqz` *command).*

1.8 Conclusions

Draw your conclusions on the exercise performed, explaining how well your estimator has performed and which are its limitations. Add suggestions of applications in which the recursive estimation should be used.

Lab 2

Kalman filter

2.1 Outline of the assignment and deliverable

In this assignment a short recap of the state-space model and of the Kalman filter is given, followed by an exercise on a simulation setup.

The exercise requires some coding in Matlab. Furthermore, you are requested to complete a report on the simulation exercise. In the report, you should summarize the main steps of the exercise, including figures and explanation of the relevant results, and provide the answers to the questions explicitly asked in the text of the exercise. The report will influence your grade for the Lab and it will guide the oral examination.

2.2 Goals

You will:

- Learn how to represent a physical system in discrete-time state-space format and implement the model obtained in Matlab and Simulink.
- Familiarize yourself with the Kalman filter by implementing it in Matlab.
- Compare different state estimation methods to the Kalman filter on the tested system.
- Understand the effects of the parameters of the Kalman filter by testing the accuracy of the state estimate for different initial values.
- Train scientific writing by delivering a report on the assignment.

2.3 Background

The Kalman filter assumes that the investigated system is described by a discrete-time linear state-space model (FIGURE 2.1), defined as:

$$x_{t+1} = Ax_t + Bu_t + v_t \qquad v_t \sim \mathcal{N}(0; R_v) \tag{2.1a}$$
$$y_t = Cx_t + Du_t + n_t \qquad n_t \sim \mathcal{N}(0; R_n) \tag{2.1b}$$

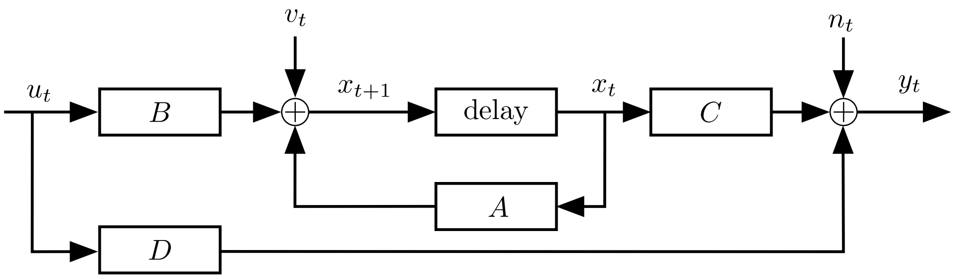

Figure 2.1: State-space equations represented as a block schematic

where:

- u_t and y_t are the input and output signals of the system at time t, respectively.
- x_t is the vector of states of the system.
- v_t and n_t are the Gaussian process noise and output (measurement) noise, with zero mean and covariance matrices R_v and R_n.
- The matrices A, B, C, D, R_v and R_n are assumed to be known.

The purpose of the Kalman filter is to estimate the evolution of the state vector x_t, based on noisy measurements of y_t. This estimate is formulated as a recursive procedure, iterating over time. The state vector is estimated at every iteration step according to the following formula:

$$\underbrace{\hat{x}_{t+1|t+1}}_{\text{Kalman state estimate}} = \underbrace{\underbrace{\hat{x}_{t+1|t}}_{\text{Predicted state}}}_{\text{Prediction}} + \underbrace{\underbrace{K_{t+1}}_{\text{Kalman gain}}(\underbrace{y_{t+1} - C\hat{x}_{t+1|t} - Du_{t+1}}_{\text{Measurement}})}_{\text{Correction}}$$

The formula consists of two parts:

- **Prediction**
  The state at the time $t+1$ is predicted based on the estimated state at the time t, the current input u_t and the system matrices:
  $$\hat{x}_{t+1|t} = A\hat{x}_{t|t} + Bu_t. \tag{2.2}$$
  The covariance of the prediction, denoted Q_{t+1}, is computed as:
  $$Q_{t+1} = AP_tA^T + R_v. \tag{2.3}$$

- **Correction**
  The predicted state is corrected with the help of the measurement. The measurement error, computed between the output at time $t+1$ and the output computed from the predicted state, is multiplied by the Kalman gain, computed at time $t+1$. The Kalman gain balances the weight between the predicted state and its correction in the state estimation and is computed as:

$$K_{t+1} = Q_{t+1}C^T(CQ_{t+1}C^T + R_n)^{-1}, \tag{2.4}$$

  starting from the matrices P_t and Q_{t+1}, representing the covariance of the estimated state at time t and the covariance of the predicted state at time $t+1$. The covariance of the corrected state is given by P_{t+1}:

$$P_{t+1} = (I - K_{t+1}C)Q_{t+1}. \tag{2.5}$$

  To start off the algorithm, initial guesses of $x_{t=0}$ and $P_{t=0}$ are required.

h

2.4 Application of the Kalman Filter to a second order system

In this part we will be dealing with a second order system in a simulation environment. First, we will obtain a state-space model of the system, from which we will simulate the output of the system. Then, we will compare different techniques to retrieve the state of the system.

The system considered is an RLC circuit and is depicted in FIGURE 2.4. The input to the system is a voltage source U_s, while the measured output is the voltage across the capacitor V_c .

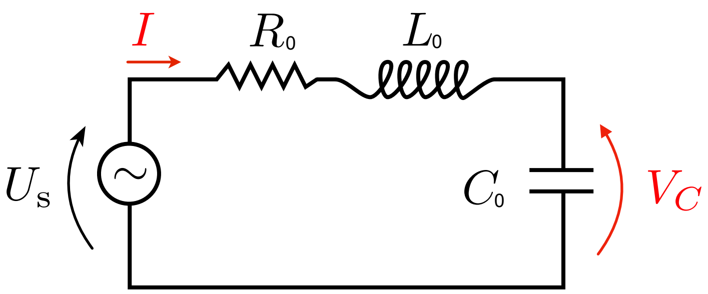

Figure 2.2: Electrical circuit composed of a voltage source (U_s), a resistor (R_0), an inductor (L_0), and a capacitor (C_0), connected in series.

State-Space model

Continuous state-space model

The system above can be represented in continuous-time state-space form, as:

$$\begin{aligned}\frac{dx(t)}{dt} &= A_c x(t) + B_c u(t)\\ y(t) &= C_c x(t) + D_c u(t);\end{aligned} \tag{2.6}$$

where the state vector $x(t)$ is $\begin{bmatrix} x_1(t) \\ x_2(t) \end{bmatrix} = \begin{bmatrix} v_c(t) \\ i(t). \end{bmatrix}$.

Task 2.1 *Obtain the matrices A_c, B_c, C_c and D_c of the state-space model representing the dynamics of the RLC circuit above, in function of the parameters R_0, L_0 and C_0. (You can start by applying Kirchoff's circuit laws in the Laplace domain). Write down the steps performed in the report.*

Matlab implementation as discrete-time state-space model

Use the matrices A_c, B_c, C_c and D_c from the previous step to implement the state-space model in Matlab. (Hint: you can use the Matlab function **ss**). Consider:

$$L_0 = 5\,\mu\text{H}, \qquad R_0 = 10\,\Omega, \qquad C_0 = 3\,\text{nF}. \tag{2.7}$$

Then, discretize the model with a zero-order hold (you can use the function **c2d**). Use the **step** command to compute and compare the step responses of the continuous- and discrete-time state-space models. Note that you should use an appropriate sample time T_s to discretize the model. The matrices you have obtained will be used to simulate the system in the next section.

Task 2.2 *Show the continuous- and discrete-time step responses, and motivate your choice for the sample time. Write down the discretized matrices A_d, B_d and C_d in the report. Is the system stable? And is it observable?*

Simulink simulation

Complete the Simulink model *ForStudents.StateSpaceModelKalmanAssignment*, representing Equation (2.1), to obtain a representation of our RLC circuit.

We consider the process and measurement noise as zero mean Gaussian signals with covariance matrices

$$R_{v_{\text{sim}}} = \begin{bmatrix} 0.025 & 0 \\ 0 & 10^{-6} \end{bmatrix}, \qquad R_{n_{\text{sim}}} = 0.0025 \tag{2.8}$$

Task 2.3 *Simulate the system, using the sampling time T_s selected above and selecting the simulation time such that your measurement will have (around) 200 points. Choose the input $u(t)$ as white Gaussian noise with standard deviation 1 V. Set (in the time-delay block) the initial condition $x_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$. Add the names of the blocks in the Simulink and add a screenshot of it in your report. Extract the true state of the system from the Simulink model.*

Note that we can extract the true state of the system only since we are working on a simulation environment. In a real life experimental setup, we would only be able to measure the input an the output signals, while we would need to estimate the state. In the remaining of the Lab, we are going to assume to be in a real life experimental setup and we are going to use system identification to obtain an estimate of the state starting from the measured input/output signals. We are going to use the true state of the system just to check the accuracy of our system identification methods.

State estimation methods

Implement the code for the state estimation using the following methods. Make sure that the variables are not hard coded, such that the methods can be used for different initial values and model parameters. Show this part of the code in your report, remembering to properly comment your steps. (You can write the code in Matlab using **`for`** loops.)

The methods to estimate the states are the following:

a Prediction only
Obtain an estimate of the state of the system, by directly applying the state-space formula: $\hat{x}_{t+1} = A\hat{x}_t + Bu_t$.

b Kalman filter: prediction + correction
Obtain the Kalman state estimate by using the formulas for the Kalman filter summarised in Section 2.3. Make sure to save as well the values of the Kalman gain K and of the covariance P in time. For $P_{t=0}$, choose $\begin{bmatrix} 100 & 0 \\ 0 & 100 \end{bmatrix}$.

Comparison of performance

Task 2.4 *Compute the state estimates using method a (prediction only) and method b (Kalman filter).*

1. *Use the correct system matrices, correct initial conditions, and correct covariance matrices.*
   $A = A_d$, $B = B_d$, $C = C_d$, $D = D_d = 0$, $\hat{x}_{t=0} = x_0$, $R_v = R_{v_{\text{sim}}}$, $R_n = R_{n_{\text{sim}}}$
2. *Use incorrect system matrices*
   $A = 2 \cdot A_d$, $B = B_d$, $C = C_d$, $D = D_d = 0$

3. *Use incorrect initial conditions*
   $\hat{x}_{t=0} = \begin{bmatrix} 0.02 \\ 0.02 \end{bmatrix}$
4. *Use incorrect measurement noise covariance*
   $R_n = 100 \cdot R_{n_{\text{sim}}}$
5. *Use incorrect process noise covaraince matrix*
   $R_v = 100 \cdot R_{v_{\text{sim}}}$

Show plots of the estimated states and the true state. Add as well a plot of the normalised difference between the true state and of the estimated states

$$\frac{\hat{x}(t) - x(t)}{\text{RMS}\{x(t)\}} \tag{2.9}$$

Task 2.5 *Comments the results in your report, making sure to provide an answer to the following questions. Which method performs better? Which method is more robust? Is there a difference in performance between the estimate of the first state (voltage capacitor) and of the second state (current) for method a? And for method b? Why? What are the effects of the covariance matrices on the Kalman gain?*

Conclusions

Task 2.6 *Draw some conclusions on what we have observed, highlighting the advantages and limitations of the Kalman filter that you have seen and comparing it to the other state estimation methods we have analyzed. Add one or two suggestions for practical situations in which you would use the Kalman filter.*

Lab 3

Non-parametric identification of a time-varying system

3.1 Goal

Obtain an approximate estimate of the time-varying FRF of an LTV system, by using a multisine excitation with a sparsely excited grid.

3.2 Context

Although LTI models are convenient to be used, their applicability is often limited. In this lab we will relax the LTI conditions, to allow the system to be time-varying. As explained in the lecture notes (Chapter 8), the dynamic behaviour of any LTV system can be described by its time-varying impulse response $g(t,\tau)$, as follows:

$$y(t) = \int_{-\infty}^{t} g(t,\tau)u(\tau)\,\mathrm{d}\tau, \tag{3.1}$$

where $u(t)$ and $y(t)$ denote the input and output signals respectively. The time-varying transfer function is computed from $g(t,\tau)$ as

$$G(j\omega,t) = \int_{0}^{\infty} g(t,t-\tau)e^{-j\omega\tau}\,\mathrm{d}\tau. \tag{3.2}$$

This time-varying transfer function can be written as a series expansion

$$G(j\omega,t) = \sum_{p=0}^{\infty} G_p(j\omega)b_p(t), \tag{3.3}$$

which allows for an intuitive interpretation, given by the block schematic in FIGURE 3.1. Namely, the input signal $u(t)$ is first filtered by the LTI blocks G_p,

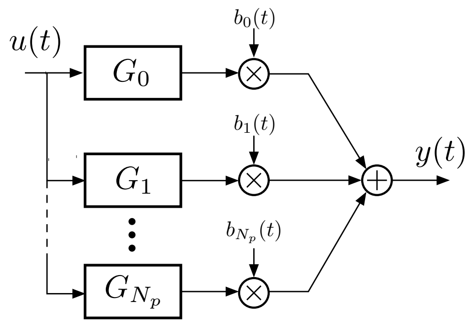

Figure 3.1: The series expansion of the time-varying transfer function, represented as a parallel block structure.

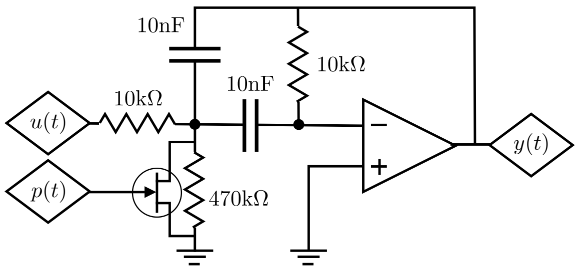

Figure 3.2: Electric schematic of the LPV circuit. The OPAMP was of the type `TL071CN`, and the transistor `BF245B`.

and then modulated by the time-varying functions $b_p(t)$. To be of practical use, this series expansion is truncated to a finite (small) number.

In this lab, we will approximate the time-varying transfer function as the series expansion (3.3), where the series is truncated after the 1st order term (i.e. $G_p = 0$ for $p > 1$), and by letting $b_p(t)$ be a pth order polynomial in t.

3.3 Considered system

The considered system is an electronic filter, the schematic of which is given in FIGURE 3.2. It is a second order bandpass filter with one variable resistance, implemented as a parallel connection of an n-type J-FET transistor (type BF245B) and a 470 kΩ resistor. The scheduling variable $p(t)$ is the gate-source voltage of the transistor. It should be negative because the transistor has an n-type depletion channel. Its resistance increases when $p(t)$ increases in amplitude.

A qualitative description of the dynamic behaviour of the circuit follows from basic derivations in electrical network theory. We have that, for a constant scheduling signal $p(t) = p^*$, the voltage transfer function $Y(s)/U(s)$ (i.e. the ratio of Laplace transforms of the output and input signals, with s the Laplace variable) has a zero at the origin, and one pair of complex conjugate poles. Only the imaginary parts of the latter depend on p^*. Lower values of $|p^*|$ yield higher values of the imaginary parts (in absolute value) of the poles and, thus, increased resonance frequencies.

This dynamic behaviour is verified with measurements with constant scheduling signals. The results are given in FIGURE 3.3. The transfer function and associated poles and zeroes are plotted, where each shade of gray corresponds to a constant value of the scheduling signal p, expressed in Volts (V). Indeed, the poles are moving along lines parallel to the imaginary axis, towards higher frequencies for lower amplitudes of p.

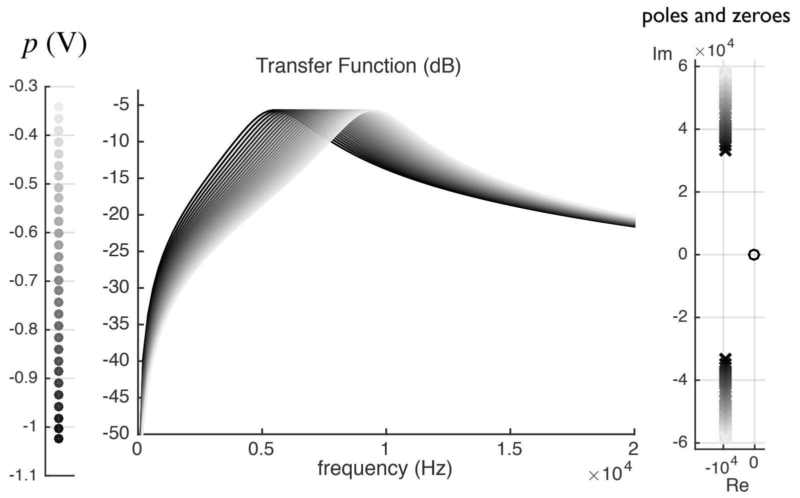

Figure 3.3: Middle plot: estimated transfer functions of the system, for different values of the constant scheduling variable (left figure). Right plot: corresponding poles x and zero o.

3.4 Methodology

The time-varying FRF will be measured, when the scheduling variable is a linearly varying function of time. The method described in Chapter 9 of the lecture notes will be used, where a first order approximation is made for the time variation (see Section 9.2.3 in the lecture notes). The methodology consists of the following sub tasks.

Task 3.1 *Design the multisine signal in the useful frequency band of the system (use* FIGURE 3.3 *for inspiration), with an RMS of about* 500 mV. *Make sure that you have at least* 30 *excited frequencies in that band, and that each pair of excited frequencies is separated by a sufficient amount of bins, such that the 'skirts' due to the time variation will be clearly visible.*

Task 3.2 *Design the scheduling signal as a linear function of time, between* $-0.4\,\mathrm{V}$ *and* $-1\,\mathrm{V}$.

Task 3.3 *Apply the multisine signal and the scheduling signal to the system (by using the VXI or Elvis systems).*

Task 3.4 *Locate the excited frequencies, and the frequencies which are 1 bin left and 1 bin right of the excited frequencies in the output spectrum.*

Task 3.5 *Use equation (9.18) from the lecture notes to compute the approximate time-varying FRF of the system. Equation (9.18) is repeated here for convenience:*

$$G(j\omega_{k_\mathrm{e}}, t) \approx \frac{-\Delta^2 Y(k_\mathrm{e}) + j\pi\big(Y(k_\mathrm{e}-1) - Y(k_\mathrm{e}+1)\big)\left(\frac{2t}{T} - 1\right)}{2U(k_\mathrm{e})} \tag{3.4}$$

where k_e *is an excited bin.*

Task 3.6 *Compare your results with an estimate of the series expansion with a higher order (e.g.* $N_p = 7$ *in* FIGURE *3.1). You can use the* `TVFRFestim` *tool from John Lataire (which can be downloaded from Canvas). Type* `help TVFRF` *to learn how to use it.*